

What Makes Agent Skills Effective in GitHub Agentic Workflows?

Anonymous Author(s)

Abstract

Agent Skills extend coding agents with reusable instructions, scripts, and references, but practitioners lack guidance on *what makes a skill good*. We study skills deployed in GitHub Agentic Workflows (gh-aw) for software maintenance—issue triage, CI investigation, fix-and-PR pipelines—and ask which intrinsic skill properties predict downstream utility, reuse, and robustness. Our approach combines repository mining of gh-aw workflows and skill artifacts with outcome metrics grounded in real maintenance workflows (e.g., pull-request merge rate and issue-triage accuracy), rather than isolated unit-test pass rates alone. We further propose a refinement loop: quantify quality, improve skills lacking key characteristics, and evaluate changes on SkillsBench [11]. We report nine research questions spanning skill quality, predictiveness, length, specificity, failure handling, modularity, decay, adoption, and static prediction of agent improvement.

CCS Concepts

• **Software and its engineering** → **Software maintenance**; • **Computing methodologies** → *Machine learning*.

Keywords

Agent Skills, GitHub Agentic Workflows, Mining Software Repositories, LLM Agents

ACM Reference Format:

Anonymous Author(s). 2026. What Makes Agent Skills Effective in GitHub Agentic Workflows?. In *Proceedings of Mining Software Repositories (MSR '27)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Coding agents increasingly rely on *Agent Skills*—packaged procedural knowledge (SKILL.md), scripts, and reference files—to perform recurring software tasks. GitHub Agentic Workflows (gh-aw) compile markdown workflow definitions into executable CI pipelines, providing a repository-scale record of how agents are deployed for maintenance work: triaging issues, investigating CI failures, proposing fixes, and opening pull requests.

Main question. Despite rapid adoption, we lack empirical answers to a central question: *what makes the skills good?* This breaks down into three sub-questions:

- (1) What does *good* mean for an Agent Skill?
- (2) Which metrics quantify skill quality?
- (3) What does it take to build a good skill in practice?

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MSR '27,

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/08
<https://doi.org/XXXXXXX.XXXXXXX>

Prior skill benchmarks emphasize task success on isolated tests [4, 11]. SkillEval [3] advances *intrinsic* evaluation (four dimensions, seven metrics), but largely analyzes SKILL.md text rather than full skill packages (scripts, references, and sibling artifacts). We focus on **skills used in agentic workflows** mined from gh-aw deployments and evaluate effectiveness using **maintenance outcomes observable in GitHub**—pull-request merge rate, issue-triage accuracy, workflow success—complementing benchmark pass rates.

We also study **consistency**: given a skill and task, does the agent produce the expected result every time? Variability may stem from task nature, agent harness, model version, or the skill itself (a generalization problem). Ultimately, we ask: *which intrinsic properties of real-world Agent Skills predict their downstream utility, reuse, and robustness?*

Contributions.

- A mining study of Agent Skills attached to public gh-aw workflows in software maintenance repositories.
- An evaluation framework combining intrinsic skill features, artifact siblings, and workflow-derived outcome metrics.
- Nine research questions (RQ1–RQ9) linking skill characteristics to performance, reuse, and predictability.
- A proposed refinement-and-evaluation loop: identify quality indicators, improve deficient skills, and validate on SkillsBench [11].

2 Motivation and Scope

Focus scope. We scope the study to **software maintenance** workflows powered by gh-aw, including:

- issue triage,
- CI failure investigation,
- automated fix-and-PR creation.

Among these tasks, practitioners attach skills to make workflows more effective. Our goal is to characterize *which skill properties correlate with that effectiveness*.

Proposed approach (Idea 1). Our planned methodology follows four steps:

- (1) **Quantify** what makes a skill good using intrinsic and extrinsic signals mined from GitHub and the GitSkills corpus [12].
- (2) **Handpick** skills that lack key quality features identified in step 1.
- (3) **Improve** those skills by adding missing characteristics (specificity, failure handling, modularity, etc.).
- (4) **Evaluate** refined skills against the SkillsBench dataset [11] and, where possible, on gh-aw workflow reruns.

This design lets us connect repository-mining insights with controlled benchmark evaluation and real workflow outcomes.

3 Background and Related Work

SkillsBench and task-based evaluation. SkillsBench [11] evaluates agent skills at scale with emphasis on task completion. Related

work on skill selection, retrieval, and adaptation [2] studies how agents choose and refine skills during execution. These approaches are valuable but often measure success via unit tests or sandboxed tasks.

Skills utility benchmarks. Utility-oriented benchmarks [4] primarily report task success rate. We differentiate by measuring maintenance effectiveness in version control: PR merge rate, triage accuracy, and workflow-run conclusions derived from gh-aw artifact logs.

SkillEval (intrinsic evaluation). SkillEval [3] compares skills along four dimensions with seven intrinsic metrics—a strong complement to task-based evaluation. We extend this line of work in two ways:

- we analyze not only SKILL.md but also *artifact siblings* (scripts, references, and bundled resources);
- we link intrinsic scores to *PR-merge-based* outcomes as a task-based subset grounded in real maintenance workflows.

GitHub Agentic Workflows and skills.

gh-aw compiles `.github/workflows/*.md` into lock files and runs agents inside GitHub Actions [9]. Skills enter a workflow through several compiler-supported mechanisms documented by GitHub [7, 8, 10]: *frontmatter skills*: (preferred for external or local installs), *inline ## skill*: blocks, *prompt-side hint* and *fusion* fragments, and (separately) *plugins*: for Agent Plugins. Copilot discovers on-disk skills from standard project directories (`.github/skills/`, `.claude/skills/`, `.agents/skills/`) and loads them on demand when the model judges them relevant [5, 6]. Each skill follows the open SKILL.md specification [1]. Workflow runs produce artifacts (activation snapshots, `aw_info.json`, Copilot process logs, agent outputs) that let us distinguish skill *declaration*, *installation*, *availability*, and *use*.

GitSkills dataset. The GitSkills corpus [12] catalogs skill artifacts across repositories with metadata on location, composition, history, and sibling files—our primary source for cross-repo intrinsic feature extraction.

4 Research Questions

We organize the study around nine research questions:

- RQ1** What characteristics distinguish high-quality Agent Skills from low-quality skills?
- RQ2** Which intrinsic characteristics predict downstream agent performance?
- RQ3** Does skill length help or hurt?
- RQ4** Does procedural specificity predict effectiveness?
- RQ5** Does failure-handling information predict robustness?
- RQ6** Does modularity predict reuse?
- RQ7** Does skill quality decay as the underlying tools/models change?
- RQ8** What characteristics predict skill reuse, forking, and adoption?
- RQ9** Can we predict whether a skill will improve an agent *without* actually executing the agent?

RQ9 is particularly important: if intrinsic features alone predict utility, practitioners could prioritize skill authoring and review before costly agent runs.

5 Methodology

5.1 Identifying gh-aw repositories

We detect gh-aw usage when a public repository contains a matching workflow pair on the default branch: `.github/workflows/{name}.md` and `.github/workflows/{name}.lock.yml`. We crawl candidates via the GitHub Code Search API with size-based query splitting to overcome the 1,000-result cap.

5.2 How gh-aw registers skills

gh-aw separates **skill installation** (compiler-managed, before the agent job) from **prompt shaping** (how skill text reaches the model). We treat these as distinct signals in our mining pipeline.

Frontmatter skills: (preferred).

The top-level skills: array in workflow frontmatter instructs gh-aw to install Copilot skills during the *activation* job, before the agent starts [8, 10]. Entries may be:

- **Local paths** such as `.github/skills/my-skill` or `skills/my-skill`, installed with `gh skill install ... -from-local`;
- **External references** `owner/repo[/path]@ref`, SHA-pinned at compile time when possible; or
- **Object entries** with per-skill `github-token` or `github-app` credentials for private skill repositories [8].

The compiler emits activation steps; authors must not add manual `gh skill install` steps in steps: or prompt text [10].

Inline, hint, and fusion strategies. Beyond skills:, gh-aw supports prompt-side skill content:

- **Inline skills** — `## skill: 'name'` blocks extracted at setup into engine skill directories [10];
- **Hint** — prompt text directing the agent to discover relevant SKILL.md files under `skills/` or `.github/skills/`;
- **Fusion** — HTML comments such as `<!-- gh-skill-fusion: path#section -->` that splice specific SKILL.md sections into the workflow prompt [10].

These strategies shape prompt content but do not replace skills: installation for external packages.

Plugins and APM imports (out of scope for core skill labels). `plugins:` installs Agent Plugins through the selected engine (e.g., `copilot plugin install`) [8, 10]. `imports:` with APM packages: can distribute skills among other agent primitives [7]. We record these declarations separately; our primary “skill” labels target SKILL.md-based Copilot skills.

5.3 How Copilot discovers skills at runtime

After gh-aw restores the checkout and any installed skill directories, the GitHub Copilot CLI engine discovers skills from configured locations [5, 6]:

- **Project skills** — `.github/skills/`, `.claude/skills/`, or `.agents/skills/` in the repository;
- **Personal skills** — `~/copilot/skills/` or `~/agents/skills/` (typically absent on GitHub Actions runners).

Copilot exposes a catalog of discovered skills to the model; each entry includes a `<name>`, `<description>`, and `<location>` tag

Table 1: Skill lifecycle stages and primary evidence in gh-aw run artifacts.

Stage	Question	Primary artifact / signal
Configured	Declared in workflow?	<code>.github/workflows/*.md</code> frontmatter <code>skills:</code> , inline <code>## skill:</code> , <code>hint/fusion</code> markers; mirrored in <code>activation/aw_info.json</code> "skills" list
Installed	Placed on disk before agent?	Activation console (gh skill <code>install ...</code>); <code>SKILL.md</code> under <code>activation/</code> ; <code>pre-agent-audit.txt</code> workspace listing
Available	Registered by Copilot?	<code>agent/sandbox/agent/logs/process-*.log</code> → <code><available_skills></code> with <code><location>project</location></code>
Invoked	Skill tool called?	Same process log → <code>tool_calls</code> where "name": "skill" and arguments name the skill
Followed	Skill instructions reflected in output?	Fingerprint phrases from <code>SKILL.md</code> in <code>agent_output.json</code> , safe-output payloads, or downstream GitHub entities

(project vs. builtin) in the `<available_skills>` block of the Copilot process log. Importantly, GitHub documents that Copilot *loads* a skill's `SKILL.md` content when it chooses to invoke the skill tool—availability in the catalog does not imply the skill body was injected or followed [5].

The workflow *task prompt* itself lives in the markdown body of `.github/workflows/*.md` (after YAML frontmatter). gh-aw assembles the final agent prompt at activation time by wrapping this body with system context (safe-outputs policy, GitHub context, MCP CLI instructions) in `aw-prompts/prompt.txt` [9]. Skill declarations in frontmatter are *not* automatically copied into `prompt.txt`; they reach the model through Copilot's skill discovery and optional `hint/fusion/inline` mechanisms above.

5.4 Operationalizing skill pickup from run artifacts

Because "skill use" is ambiguous in logs, we operationalize five *non-equivalent* stages (Table 1). Each stage uses specific artifacts extracted from workflow runs via the GitHub Actions API (jobs, console logs, and `activation/agent` artifact bundles).

Static configuration (Stages 1–2). For repository mining, we scan workflow markdown for a column-zero `skills:` key in YAML frontmatter (after the gh-aw collection header when present) and for inline `## skill:` headers [10]. At run time, `activation/aw_info.json` records frontmatter-configured install paths (e.g., `[".github/skills/say-hello"]`) but *not* the full Copilot catalog or builtin skills. Activation job logs confirm installation commands; the activation artifact may snapshot installed `SKILL.md` files under `activation/.github/skills/.artifacts/agent/pre-agent-`

`audit.txt` lists workspace skill paths immediately before the agent job starts.

Engine catalog and tool calls (Stages 3–4).

The authoritative runtime catalog is the Copilot CLI process log (`artifacts/agent/sandbox/agent/logs/process-*.log`). We parse the `<available_skills>` XML emitted in the skill tool definition and system prompt, retaining skill names whose `<location>` is project (user/repo skills) as distinct from builtin Copilot skills. To detect *invocation*, we parse JSON `tool_calls` blocks in the same log; a skill is invoked when `function.name == "skill"` and the arguments reference the skill name (e.g., `{"skill": "say-hello"}`). Downstream safe-output tool calls (e.g., `safeoutputs-create-issue`) are recorded separately and do not indicate skill use.

Behavioral compliance (Stage 5 / Followed). Invocation still does not guarantee compliance. When `SKILL.md` contains testable requirements (mandatory phrases, tool sequences, forbidden actions), we scan `agent_output.json` and applied safe-output payloads for those fingerprints. In our pilot run, `say-hello` was configured, installed, and *available*, but never invoked; the created issue body omitted required phrases from the skill file—illustrating why we report availability and use rates separately.

Joining with GitSkills. The GitSkills corpus [12] supplies cross-repository `SKILL.md` features and sibling artifacts. We join on repository name to enrich configured skills with intrinsic metrics, but we do *not* treat co-located GitSkills files as evidence of runtime use; only run-artifact stages above establish pickup during a specific workflow execution.

5.5 Feature extraction

From GitSkills and mined workflow files we extract intrinsic features, including:

- `SKILL.md` body length, frontmatter validity, procedural specificity;
- presence of failure-handling guidance;
- modularity (scripts, references, sibling count and bytes);
- adoption signals (forks, reuse across repos, commit history).

5.6 Outcome metrics

We link skills to downstream signals where data permits:

- **PR merge rate** — task-based maintenance success;
- **Issue-triage accuracy** — label/assignment correctness when ground truth is available;
- **Workflow conclusion** — success/failure of gh-aw runs;
- **Consistency** — variance of outcomes for the same skill—task pairing across runs, models, and repositories.

5.7 Refinement and evaluation loop

For skills identified as low quality on intrinsic metrics, we manually or semi-automatically add missing characteristics, then evaluate:

- on SkillsBench [11] (controlled benchmark), and
- on gh-aw workflow reruns where feasible (real maintenance context).

6 Study Design

Cohort. We combine (i) periodic gh-aw workflow crawls, (ii) GitSkills artifact records, and (iii) workflow-run extractions for repositories that declare skills in frontmatter.

Analysis plan. For RQ1–RQ2 we correlate intrinsic features with outcome metrics using regression and ranking models. RQ3–RQ6 test individual hypotheses (length, specificity, failure handling, modularity). RQ7 tracks feature–outcome stability across engine/model versions recorded in run artifacts. RQ8 models adoption from repository and commit-graph signals. RQ9 trains predictors using intrinsic features only, validated against held-out execution outcomes.

7 Expected Results and Discussion

We expect intrinsic metrics from SkillEval-style dimensions [3] to partially predict maintenance outcomes, but sibling artifacts (scripts, references) to add explanatory power beyond SKILL.md text alone. We also expect high variance in consistency (RQ7, generalization) driven by harness and model changes rather than skill content alone. RQ9 will indicate whether static review can reduce the need for expensive agent executions during skill authoring.

8 Threats to Validity

Construct validity. “Good” spans intrinsic quality and downstream utility; we operationalize both but cannot capture all practitioner intent. Skill *availability* (Copilot catalog) and *use* (tool invocation or output fingerprints) measure different constructs; conflating them would over-estimate skill effectiveness [5].

External validity. Public gh-aw repositories may not represent private deployments; default-branch search misses branch-only workflows.

Internal validity. Confounders include repository domain, workflow template, base model, and GitHub Actions environment.

Selection bias. Skills declared in frontmatter are rare today; early adopters may not generalize to future mainstream usage.

9 Conclusion

We outlined a mining study to answer *what makes Agent Skills good* in the context of GitHub Agentic Workflows for software maintenance. By linking intrinsic skill properties—including artifact siblings—to maintenance outcomes such as PR merge rate and triage accuracy, and by validating skill refinements on SkillsBench, we aim to bridge intrinsic evaluation and real-world agentic workflow effectiveness. Future work includes scaling live crawls, expanding run-artifact linkage, and releasing reproducible feature and outcome datasets.

References

- [1] Agent Skills Contributors. 2026. Agent Skills Specification. <https://github.com/agentskills/agentskills>. Open SKILL.md format used by Copilot and gh-aw.
- [2] Anonymous. 2026. Evaluating Skill Selection, Retrieval, and Adaptation for Coding Agents. *arXiv preprint arXiv:2604.04323* (2026). <https://arxiv.org/pdf/2604.04323>
- [3] Anonymous. 2026. SkillEval: Intrinsic Evaluation of Agent Skills. *arXiv preprint arXiv:2608.06891* (2026). <https://arxiv.org/pdf/2608.06891> Four dimensions, seven intrinsic metrics.
- [4] Anonymous. 2026. Skills Utility Benchmarks for Task Success Rate. *arXiv preprint arXiv:2606.11435* (2026). <https://arxiv.org/pdf/2606.11435>
- [5] GitHub. 2026. About Agent Skills. <https://docs.github.com/en/copilot/concepts/agents/about-agent-skills>. Project vs. personal skill locations; on-demand loading.
- [6] GitHub. 2026. Adding Agent Skills for GitHub Copilot CLI. <https://docs.github.com/en/copilot/how-to/copilot-cli/customize-copilot/add-skills>. `.github/skills`, `.claude/skills`, `.agents/skills`, and `copilot skill` commands.
- [7] GitHub. 2026. gh-aw Frequently Asked Questions. <https://github.github.com/gh-aw/reference/faq/>. Skill installation vs. imports and APM packages.
- [8] GitHub. 2026. gh-aw Workflow Frontmatter Reference. <https://github.github.com/gh-aw/reference/frontmatter/>. Documents `skills:`, `plugins:`, and related fields.
- [9] GitHub. 2026. GitHub Agentic Workflows (gh-aw). <https://github.com/github/gh-aw>. Agentic workflow compiler and CLI.
- [10] GitHub. 2026. Skills in Agentic Workflows. <https://github.com/github/gh-aw/blob/main/.github/aw/skills.md>. Compiler-managed `skills:plugins:`, `hint`, `fusion`, and `inline` strategies.
- [11] SkillsBench Team. 2026. SkillsBench: Evaluating Agent Skills at Scale. <https://www.skillsbench.ai/skillsbench.pdf>. Accessed: 2026-08-23.
- [12] Mario Vaccargiu et al. 2026. GitSkills: A Dataset of Agent Skill Artifacts from GitHub. Hugging Face dataset <https://huggingface.co/datasets/mvaccargiu/gitkills>. Artifacts, siblings, repos, mining runs.